# Homework 4

## Total Points: 100 (Correctness and Style) + 3 EC (Checkpoint)

**Due Dates (SD time):**

- **Entire Assignment: Monday, April 28th, 11:59pm**
- **Checkpoint (read below): Saturday, April 26th, 11:59pm**

## Starter Files

Download hw04.zip. Inside the archive, you will find starter files for the questions of this homework. You cannot import anything to solve the problems.

## IMPORTANT: Coding and Docstring Style

This is a reminder that your code style will be graded. Here are a few useful links:

Style Guide on Course Website

Style Guide Examples

## Testing

At any point of the homework, use one of the following command to test your work:

```
>>> python3 -m doctest hw04.py
>>> py -m doctest hw04.py
>>> python -m doctest hw04.py
```

## Checkpoint Submission

### Due Date: Saturday, April 26th, 11:59pm (SD time)

You can earn up to **3 points extra credit** by submitting the checkpoint by the due date above. In the checkpoint submission, you should complete **Question 1, 2, 3** and submit the hw04.py file to gradescope.

Checkpoint submission is graded by completion, which means you can get full points if your code can pass some simple sanity check (no tests against edge cases). Note that in your final submission, you should still submit these questions, and you may modify your implementation if you noticed any errors.

## General Notes and Requirements

1. **DO NOT IMPORT ANY PACKAGES.**
2. Please add your own doctests (**at least three**) as the given doctests are not sufficient. **You will be graded on the doctests**.
3. When a question **(Questions 4, 5)** requires **assert statements**: follow the assert guide to write assert statements to prevent any unexpected inputs. You will be graded on this. If a question does not require assert statements, assume all inputs are valid.

## Required Questions

The competition is over, and the winning team now has a chance to play in the city championship. Let's prepare for it!

## Question 1:

You have a file containing the players' information, but it needs to be cleaned.

The file includes:

- A header line
- Then, each line contains the name, city, and date of birth (all lines, including the header, are separated by a comma followed by a space).

**Note**: The name, city, and/or date of birth may have leading or trailing spaces due to errors made by the file's creator, and you need to remove them as well. See the doctests for details.

Write a function that takes the path to a .txt file (as a string) and returns:

- a dictionary, where:
  - the keys are the city names and
  - the values are lists of people born there.
- an empty dictionary if the file contains only the header line.

Your code should be case-sensitive (i.e., 'Chicago' and 'chicago' are considered different).

**Note**: **No** assert statements needed.

**Assume:**

1. There are no empty strings in place of the name, city, or date of birth (DOB).
2. The file always exists and will not be empty (it will contain at least the header).
3. Only remove leading and trailing whitespaces, not punctuation.
4. If you define a variable for the date of birth, please name it dob (lowercase) to conform to the style guide

```
def place_of_birth(file_in):
    """
    >>> place_of_birth('files/info_1.txt')
    {'Chicago': ['Rob'], 'New York': ['Ella'], 'New York.': ['Mary']}
    >>> place_of_birth('files/info_2.txt')
    {'Chicago': ['Rob'], 'London': ['Ezra'], 'Paris': \
['Mary'], 'paris': ['Ron', 'Harry']}
    >>> place_of_birth('files/header.txt')
    {}
    """
```

## Question 2:

Let's continue!

You have a file containing the players' information, but it needs to be cleaned.

The file includes:

- A header line
- Then, each line contains the name, city, and date of birth (all lines, including the header, are separated by a comma followed by a space).

**Note**: The name, city, and/or date of birth may have leading or trailing spaces due to errors made by the file's creator, and you need to remove them as well. See the doctests for details.

Write a function that takes two parameters:

- the path to a .txt file (as a string) and
- the path to an output file (as a string)

The purpose of this function is to categorize people into three groups: older than 35, younger than 35, or exactly 35 years old. You may use only the year to determine age (for simplicity). The function should not return anything. If the input file contains only the header, the output file should also contain only the header.

**Information that needs to be written to a new file (comma separated, no space):**

- File header: `name,older than 35`
- On each line: name, 1 if older than 35, -1 if younger than 35, and 0 if exactly 35.

**Example:**

| Input Line | Output Line | Reason |
|---|---|---|
| Tracy, San Diego, 10/10/2000 | Tracy,-1 | 2024 - 2000 = 24 which is less than 35 |

**Note**: **No** assert statements needed.

**Assume:**

1. There are no empty strings in place of the name, city, or date of birth (DOB).
2. The file always exists and will not be empty (it will contain at least the header).
3. Only remove leading and trailing whitespaces, not punctuation.
4. The current year is 2024.
5. You may have an extra line at the end of the output file.
6. If you define a variable for the date of birth, please name it dob (lowercase) to conform to the style guide.

```
def age_groups(file_in, file_out):
    """
    >>> age_groups('files/info_1.txt', 'files/age_1_out.txt')
    >>> with open('files/age_1_out.txt', 'r') as outfile1:
    ...     for line in outfile1:
    ...         print(line.strip())
    name,older than 35
    Rob,-1
    Ella,1
    Mary,-1
```

```
>>> age_groups('files/info_2.txt', 'files/age_2_out.txt')
>>> with open('files/age_2_out.txt', 'r') as outfile2:
...     for line in outfile2:
...         print(line.strip())
name,older than 35
Rob,-1
Ezra,1
Mary,1
Ron,0
Harry,0

>>> age_groups('files/header.txt', 'files/empty_out.txt')
>>> with open('files/empty_out.txt', 'r') as outfile:
...     for line in outfile:
...         print(line.strip())
name,older than 35
"""
```

## Question 3:

Finally let's work with multiple files at once!

**Now each** file includes:

- A header line
- Then, each line contains the name, city, and date of birth (all lines, including the header, are separated by a comma followed by a space).

**Note**: The name, city, and/or date of birth may have leading or trailing spaces due to errors made by the file's creator, and you need to remove them as well. See the doctests for details.

Write a function that takes two parameters:

- a list of paths to .txt files (as a list of strings) and
- a path to an output file (as a string).

The purpose of this function is to read all the files from the list, modify some information (as described below), and combine them into a single output file. The function should not return anything.

**<u>Modification</u>:** The date of birth information (after removing any whitespaces) is in the format MM/DD/YYYY (with no spaces between components). You need to convert the date to a new format, adding spaces between each component:

`Month (name) DD YYYY`

For example, if the date is 10/21/2020, the new format would be 'Oct 21 2020.' Use the first three letters of each month.

**Information that needs to be written to a new file (comma separated, no space):**

- File header: `name,city,DOB`
- On each line: name, city, modified date of birth

**Note**: **No** assert statements needed.

**Assume:**

1. There are no empty strings in place of the name, city, or date of birth (DOB).
2. Every file to be read always exists and will not be empty (it will contain at least the header).
3. Only remove leading and trailing whitespaces, not punctuation.
4. You may have an extra line at the end of the output file.
5. If you define a variable for the date of birth, please name it dob (lowercase) to conform to the style guide.

**Hint**: Create a helper dictionary with month number <-> name correspondence. You don't need to define magic numbers inside the helper dictionary.

```
def several_files(files_lst, file_out):
    """
    >>> lst_1 = ['files/info_1.txt','files/info_3.txt',
'files/info_4.txt']
    >>> several_files(lst_1, 'files/several_1_out.txt')
    >>> with open('files/several_1_out.txt', 'r') as outfile1:
    ...     for line in outfile1:
    ...         print(line.strip())
    name,city,DOB
    Rob,Chicago,Oct 10 2010
    Ella,New York,Apr 09 1970
    Mary,New York.,Jan 01 2004
```

```
Sue,San Diego,Mar 19 2015
Ben,London,Dec 08 1970
Kate,Paris,Jul 13 1945

>>> lst_2 = ['files/info_2.txt','files/header.txt']
>>> several_files(lst_2, 'files/several_2_out.txt')
>>> with open('files/several_2_out.txt', 'r') as outfile2:
...    for line in outfile2:
...       print(line.strip())
name,city,DOB
Rob,Chicago,Oct 10 2010
Ezra,London,Apr 12 1978
Mary,Paris,Sep 11 1975
Ron,paris,Nov 11 1989
Harry,paris,Dec 15 1989
"""
```

## Question 4:

The team is away for the championship but misses their co-workers so much that they want to send them a few postcards. They have been given a list of tuples, where each tuple is of the following form:

```
(
    name (as string),  # assume with first and last name separated by one space
    price (as non-negative integer),
    age (as non-negative integer),
    place (as string)
)
```

Write a function that generates postcards as strings only for prices strictly **lower than 75**. The output must be a dictionary with names as keys and postcards as their corresponding values. Each postcard should include the following components, **all in lowercase**:

- The first three letters of the person's first name (if the first name has fewer than three letters, use the entire name)
- The age
- The person's full last name

- The last digit of the price
- The *reversed* place

**Requirements:**

1. **Assert statements** (you may only assert the list of tuples and the info listed in the parentheses assume the string content).
2. You **cannot** use loops or list/set/dictionary comprehension for this question. Instead, you **must** use lambda and map and/or filter functions. There are no restrictions on the number of lines.

**Notes:**

1. You can assume that there will be no duplicate names in the inputs, as this could cause issues with duplicate keys in the output dictionary.
2. You can create a dictionary using `dict()` without a loop if you have a list of length-2 tuples, one for the key and another for the value
3. The zip() function might be useful when creating the list of tuples above.
4. For extra practice, you might consider implementing the function using for/while loops and/or list comprehensions.

```
def postcards(info_list):
    """
    >>> postcards([
    ...     ('Yue Wang', 96, 18, 'Hoover Dam'),
    ...     ('Cleo Patra', 10, 32, 'Bellagios')
    ... ])
    {'Cleo Patra': 'cle32patra0soigalleb'}
    >>> postcards([])
    {}
    >>> postcards([
    ...     ('Mari Noh', 155, 18, 'tram'),
    ...     ('Gwen Am', 34, 54, 'Venetian'),
    ...     ('Freya Dog', 34, 1, 'The Strip')
    ... ])
    {'Gwen Am': 'gwe54am4naitenev', 'Freya Dog': 'fre1dog4pirts eht'}
    """
```

## Question 5:

The files for the championship have been accepted, and the committee has sent you new information regarding the team point totals in each game. Given a list of integers and a sequence of possible outcomes (a list of tuples), you need to apply all the specified outcomes to the list in the given order and return the transformed list. Your function should execute the operations **in order**.

1. **advance**: `('advance', amount)`: Adds `amount` to each number in a list. Returns a list.
2. **lost**: `('lost', amount)`: Subtracts `amount` from each number in a list. Returns a list.
3. **tie:** `('tie', threshold)`: Remove all numbers less than a threshold. Returns a list.
4. **eliminate:** `('eliminate', symbol)`: `symbol` should be added to the output string:
   - If a number is positive, return symbol + 'won'. Otherwise, return symbol + 'lost'.

   Returns a list of strings.
5. **win**: `('win', message)`: `message` should be added to the output string representing the total amount of points. Returns a string.

**Input Assumptions:**

- You may assume that the provided operations would never cause any error, and the list will never be empty at any point during the execution.
- **'eliminate'** and **'win'** will always be the last step because they do not return a list of numbers.

**Requirements:**

1. You **must** fill a dictionary of `{name: lambda function}`, at the start of the given code. To call a function `name`, simply use `d[name](...)` (think about why this works).
   a. The dictionary should take the following format
   ```
   {
       Name1: lambda function,
       Name2: lambda function
       ...
   }
   ```
2. You **cannot** use list comprehensions within any of your lambda functions. **Must use lambda and map and/or filter functions only.**
3. **Explicit loops are ALLOWED <u>after</u> defining the lambda dictionary.**
   a. To run your lambda function, you should use `dict[name](params)`

4. **Assert Statements Required For Input.** List comprehension is **allowed** for assert statements.

**Assert Statement Hints:**

1. Your function should not accept empty lists.
2. Your function should not accept invalid operations not mentioned above. This check is **case sensitive.**
3. Additional assert statements may be needed.

**Example:**

```
starting_list = [1, 2, 3, 4, 5]
operations = [('lost', 3), ('tie', -1), ('win', 'game over! ')]
```

| Operation | Outputs (intermediate and final) | Reason |
|---|---|---|
| ('lost', 3) | [-2, -1, 0, 1, 2] | For 'lost' we subtract 3 for every number |
| ('tie', -1) | [-1, 0, 1, 2] | For 'tie' we eliminate everything less than -1 |
| ('win', 'game over! ') | game over! 2 | game over! is concatenated to 2 (the sum of all numbers) |

**Notes:**

1. You may assume that the provided operations would never cause any error, and the list will never be empty at any point during the execution.
2. Create one function at the time and test it, before moving on to the next one.

```
def win_or_lose(lst, operations):
    """
    >>> lst = [1, 12, 123, 1234, 12345, 123456]
    >>> operations_1 = [('advance', 5), ('lost', 3), ('tie', 4)]
    >>> win_or_lose(lst, operations_1)
    [14, 125, 1236, 12347, 123458]
    >>> operations_2 = [('lost', 200), ('eliminate', 'Team ')]
    >>> win_or_lose(lst, operations_2)
    ['Team lost', 'Team lost', 'Team lost', 'Team won', 'Team won',
'Team won']
    """
```

## Submission

When submitting to Gradescope, you can include the files folder by compressing it with your hw04.py file, creating a .zip file you can submit that zip file to Gradescope. If you have your hw04.py and files folder inside of another folder, make sure that you select hw04.py and the files folder directly to compress it, rather than compressing the encompassing folder, as it will cause issues with your submission.

You can do this by going into File Explorer or Finder and selecting **both** the files folder and the hw04.py file. Right-click and then select either

**Windows**: Send to -> Compressed (zipped) folder
**Mac**: Compress 2 items

Your .zip file should have the following files if you open it:

```
files/
--- info_1.txt
--- info_2.txt
--- ... (other files)
hw04.py
```

Please submit the homework via Gradescope. You may submit more than once before the deadline, and only the final submission will be graded. Please refer to the Gradescope FAQ if you have any issues with your submission.